

CSA09 – Programming in Java

CO3: Exception Handling and Multithreading

Individual Assignment

Question No. 23

Title

Creating and Running Multiple Threads in Java

STUDENT DETAILS

Name: ____N.SATHWIK_____

Register Number: ____192525077_____

Course Code: CSA0926

Course Name: Programming in Java

CO: CO3 – Exception Handling and Multithreading

Faculty Name: ____DR.RAM KUMAR_____

Date of Submission: _07-08-2026_____

AIM

To create three separate threads in Java, start them simultaneously, and observe that their outputs are interleaved due to concurrent execution.

OBJECTIVE

The objective of this program is to understand the concept of multithreading in Java. The program demonstrates how multiple threads execute independently at the same time. It also helps understand thread scheduling, concurrent execution, and the difference between sequential and parallel execution.

PROBLEM STATEMENT

Create three separate threads, where each thread prints its own identifying message five times. Start all three threads at approximately the same time and observe that their outputs are interleaved unpredictably. This demonstrates concurrent execution in Java.

INTRODUCTION

Java is a powerful programming language that supports multithreading. A thread is the smallest unit of execution within a process. Unlike a normal sequential program, a multithreaded program can perform multiple tasks simultaneously.

Multithreading improves the performance of applications because different tasks can execute at the same time. Java provides built-in support for threads through the **Thread** class and the **Runnable** interface.

When multiple threads are started, the Java Virtual Machine (JVM) scheduler decides which thread gets CPU time. Therefore, the order of execution cannot be predicted. This behavior is called concurrent execution.

THEORY

WHAT IS A THREAD?

A thread is an independent path of execution inside a program. Every Java application has at least one thread called the **Main Thread**.

A program can create additional threads to perform multiple tasks simultaneously.

Example:

- Downloading files
- Playing music
- Running animations
- Processing data

All these tasks can execute at the same time using threads.

WHAT IS MULTITHREADING?

Multithreading is the process of executing multiple threads concurrently within a single program.

Advantages include:

- Faster execution
- Better CPU utilization
- Improved application performance
- Ability to perform multiple operations simultaneously
- Better responsiveness

CREATING THREADS IN JAVA

Java provides two methods for creating threads.

METHOD 1: EXTENDING THREAD CLASS

class MyThread extends Thread

```
{  
    public void run()  
    {  
        System.out.println("Thread Running");  
    }  
}
```

METHOD 2: IMPLEMENTING RUNNABLE INTERFACE

class MyTask implements Runnable

```
{  
    public void run()  
    {  
        System.out.println("Thread Running");  
    }  
}
```

Runnable is preferred because Java supports only single inheritance.

Thread Life Cycle

A thread passes through several states.

1. New
2. Runnable
3. Running
4. Waiting / Timed Waiting
5. Terminated

THREAD SCHEDULER

The Thread Scheduler is a part of the JVM.

Its responsibilities include:

- Selecting the next thread
- Allocating CPU time
- Managing concurrent execution

Because scheduling is handled by the JVM and operating system, the output order is unpredictable.

ALGORITHM

Step 1: Create a class named **MyTask** implementing **Runnable**.

Step 2: Override the **run()** method.

Step 3: Print the thread name five times.

Step 4: Pause the thread for 200 milliseconds using **Thread.sleep()**.

Step 5: Create three Thread objects.

Step 6: Assign different names to the threads.

Step 7: Start all three threads using **start()**.

Step 8: Print a message from the main thread.

Step 9: Observe that the output order changes every execution.

PROGRAM

```
class MyTask implements Runnable
{
    @Override
    public void run()
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println(Thread.currentThread().getName())
```

```
    + " - Count : " + i);
```

```
    try
    {
        Thread.sleep(200);
    }
    catch(InterruptedExcepion e)
    {
        System.out.println(e);
    }
}
}
```

```
public class MultipleThreadsDemo
{
    public static void main(String[] args)
    {
        Thread t1 = new Thread(new MyTask(),"Thread-1");
        Thread t2 = new Thread(new MyTask(),"Thread-2");
        Thread t3 = new Thread(new MyTask(),"Thread-3");

        t1.start();
        t2.start();
        t3.start();

        System.out.println("Main Thread Continues...");
    }
}
```

SAMPLE OUTPUT

Main Thread Continues...

Thread-2 - Count : 1

Thread-1 - Count : 1

Thread-3 - Count : 1

Thread-2 - Count : 2

Thread-3 - Count : 2

Thread-1 - Count : 2

Thread-3 - Count : 3

Thread-1 - Count : 3

Thread-2 - Count : 3

Thread-1 - Count : 4

Thread-2 - Count : 4

Thread-3 - Count : 4

Thread-3 - Count : 5

Thread-1 - Count : 5

Thread-2 - Count : 5

Note: The output order changes every time the program runs because the JVM scheduler decides which thread executes first.

CODE SCREEN SHOT:

```
1 import java.util.*;
2 public class MultipleThreadsDemo
3 {
4     public static void main(String[] args)
5     {
6         Thread t1 = new Thread(new MyTask(),"Thread-1");
7         Thread t2 = new Thread(new MyTask(),"Thread-2");
8         Thread t3 = new Thread(new MyTask(),"Thread-3");
9
10        t1.start();
11        t2.start();
12        t3.start();
13
14        System.out.println("Main Thread Continues...");
15    }
16 }
17
18 class MyTask implements Runnable
19 {
20     @Override
21     public void run()
22     {
23         for(int i=1;i<=5;i++)
24         {
25             System.out.println(Thread.currentThread().getName()
26                 + " - Count : " + i);
27
28             try
29             {
30                 Thread.sleep(200);
31             }
32             catch(InterruptedException e)
33             {
34                 System.out.println(e);
35             }
36         }
37     }
38 }
39
```

PROGRAM EXPLANATION

The program defines a class called **MyTask** that implements the **Runnable** interface. The **run()** method contains a loop that prints the current thread name and a counter value five times.

Inside the loop, **Thread.sleep(200)** pauses the thread for 200 milliseconds. This allows the scheduler to switch between threads, making concurrent execution easier to observe.

In the **main()** method, three Thread objects are created:

- Thread-1
- Thread-2
- Thread-3

Each thread is associated with the same Runnable object.

When **start()** is called, the JVM creates three separate execution paths.

The main thread immediately prints:

Main Thread Continues...

Meanwhile, the three worker threads execute simultaneously. Since the scheduler decides which thread gets CPU time, the output appears in a different order during each execution.

CONCEPTS USED

- Multithreading
- Runnable Interface
- Thread Class
- run() Method
- start() Method
- Thread.sleep()
- Concurrent Execution
- Thread Scheduler

ADVANTAGES OF MULTITHREADING

- Improves application performance.
- Allows multiple tasks to execute simultaneously.
- Better CPU utilization.
- Reduces waiting time.
- Increases application responsiveness.
- Efficient resource sharing.
- Useful in real-time applications.

Applications of Multithreading

- Online banking systems
- Web servers

- Video streaming applications
- Multiplayer games
- Mobile applications
- Operating systems
- Download managers
- Chat applications
- File transfer software
- Scientific simulations

EXPECTED RESULT

The program successfully creates three separate threads that execute concurrently. Each thread prints its own identifying message multiple times. The output is interleaved because thread scheduling is controlled by the JVM, demonstrating concurrent execution.

LEARNING OUTCOMES

After completing this assignment, the student is able to:

- Understand the concept of multithreading.
- Create threads using the Runnable interface.
- Start multiple threads simultaneously.
- Observe concurrent execution.
- Understand thread scheduling.
- Use Thread.sleep() to control execution timing.
- Explain why thread execution order is unpredictable.
- Differentiate between sequential and concurrent execution.

VIVA QUESTIONS AND ANSWERS

1. What is a thread?

A thread is the smallest unit of execution inside a program.

2. What is multithreading?

Multithreading is the execution of multiple threads simultaneously.

3. Which method starts a thread?

The **start()** method.

4. Which method contains the thread code?

The **run()** method.

5. What is the purpose of Runnable?

It provides the task that a thread executes.

6. Why is Runnable preferred over extending Thread?

Because Java supports single inheritance, implementing Runnable allows the class to extend another class if needed.

7. What does Thread.sleep() do?

It pauses the current thread for a specified time.

8. Why does the output change every execution?

Because the JVM scheduler decides the order in which threads run.

9. Can multiple threads access the same object?

Yes, they can share the same object.

10. What is concurrent execution?

Concurrent execution means multiple threads make progress during the same period, with the scheduler switching between them.

CONCLUSION

This assignment successfully demonstrates how to create and execute multiple threads in Java using the **Runnable** interface. Three threads are started at nearly the same time, and their outputs are interleaved because the JVM scheduler controls thread execution. The program illustrates the fundamental concepts of multithreading, including thread creation, concurrent execution, thread scheduling, and the use of **Thread.sleep()**. This experiment provides a strong foundation for understanding more advanced multithreading concepts such as synchronization, inter-thread communication, and concurrent programming in Java.